Group 1:2

Link to the slides

Jim Bergdahl & Hugo Auzanot

RRT* Implementation

- **“Fake kNN”**: Code searches for all nodes within a distance (15 units)
  - Optimize path by selecting the lowest cost node
  - Then check local neighbors to find potential new nodes with lower cost
  - → Shortest path

More about RRT*

RRT* finds shortest path over time:

- Goal bias (10%)
- Updates costs for children recursively

Some kinematic constraints:

- Tree can’t expand backwards relative to car’s orientation
- … but more to be implemented.

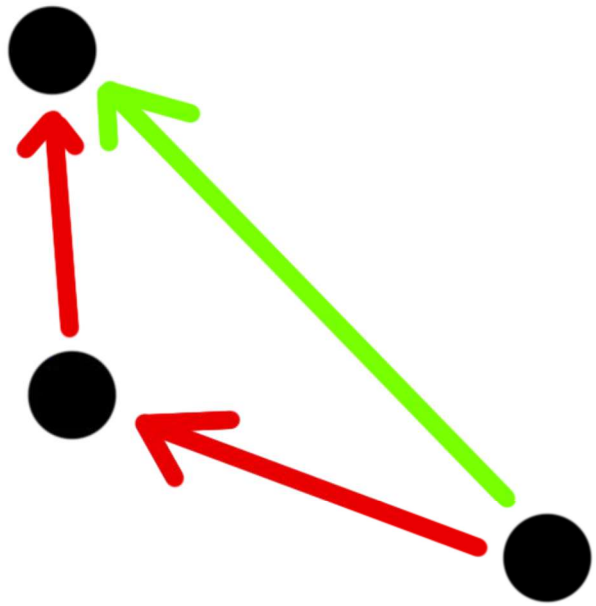
Path smoothing - what we do

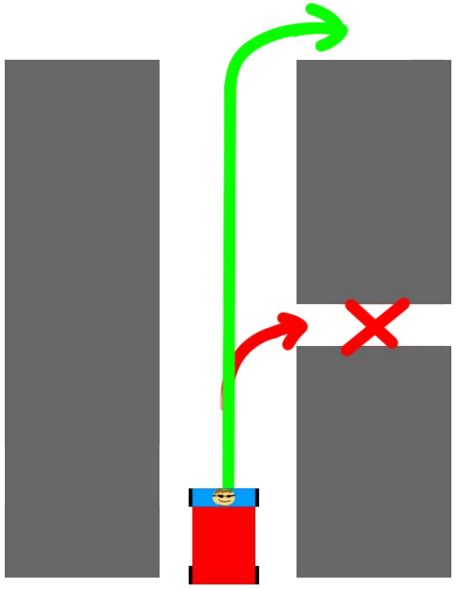
A simple SmoothPath method:

- If A connects to C without collision, remove point B.

IsTraversable method:

- Projects car's box collider along the path to ensure it actually fits.





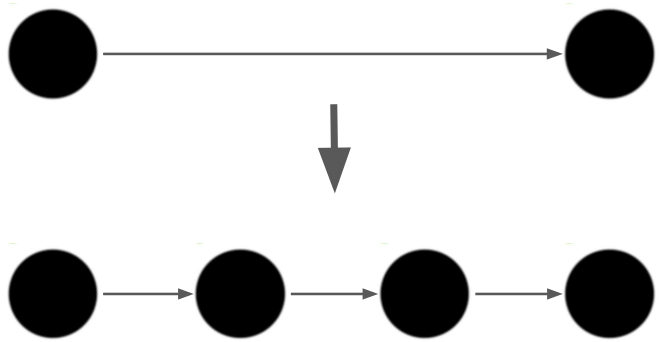
Path smoothing

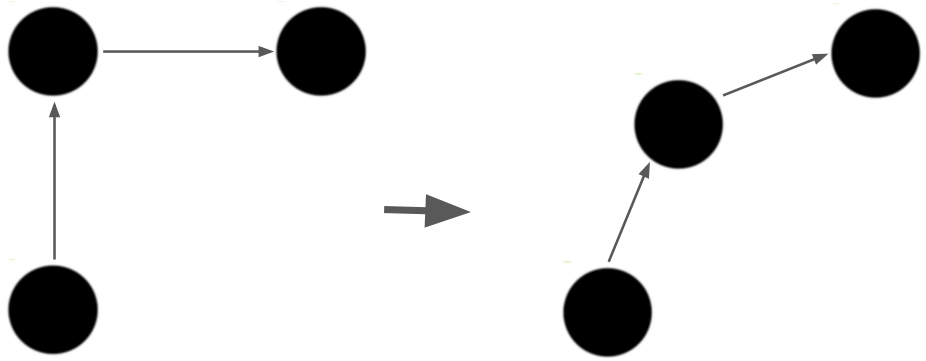
Interpolation:

- New nodes are added between each existing nodes

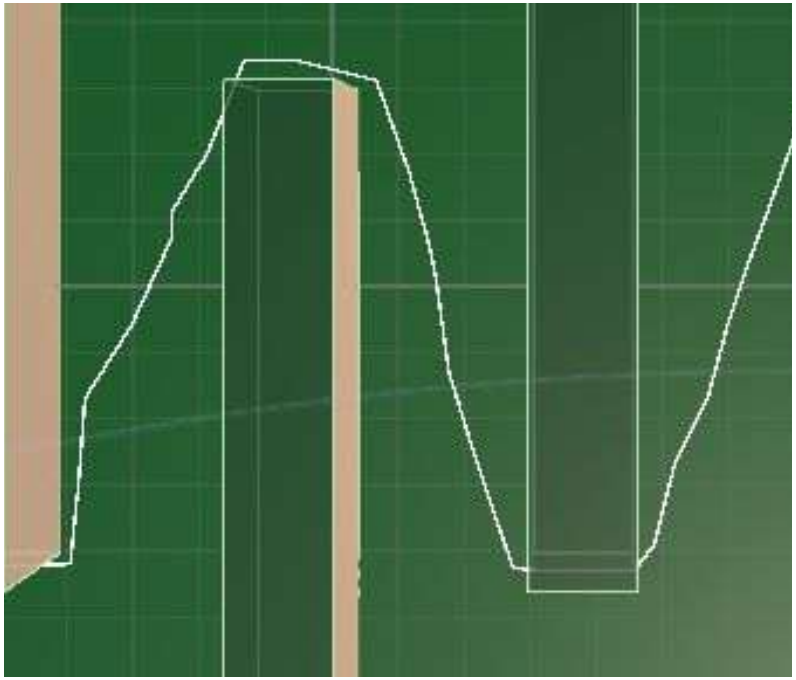
Averaging:

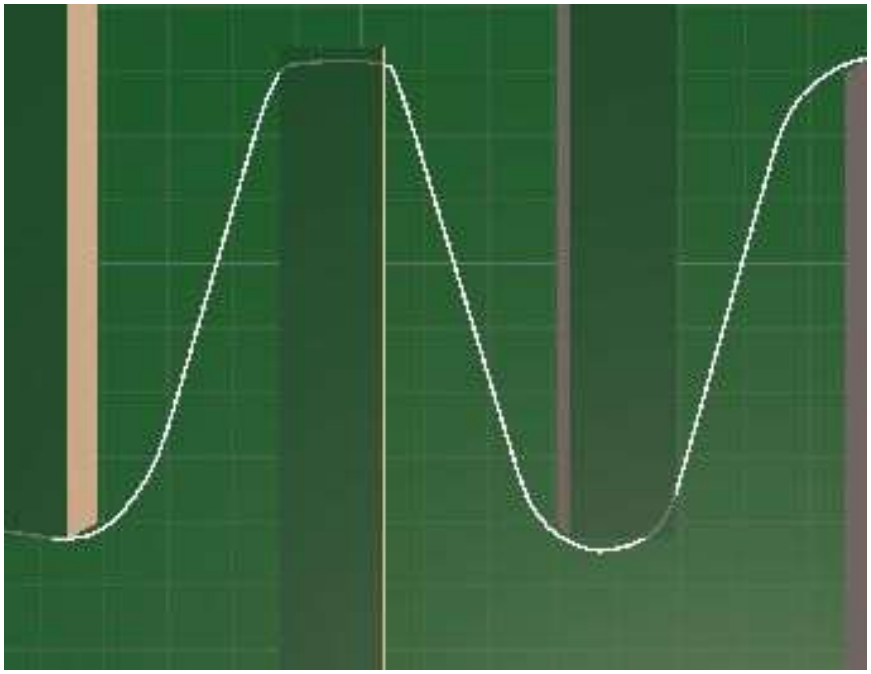
- Weighted sum between a point and its four nearest neighbors





Path smoothing



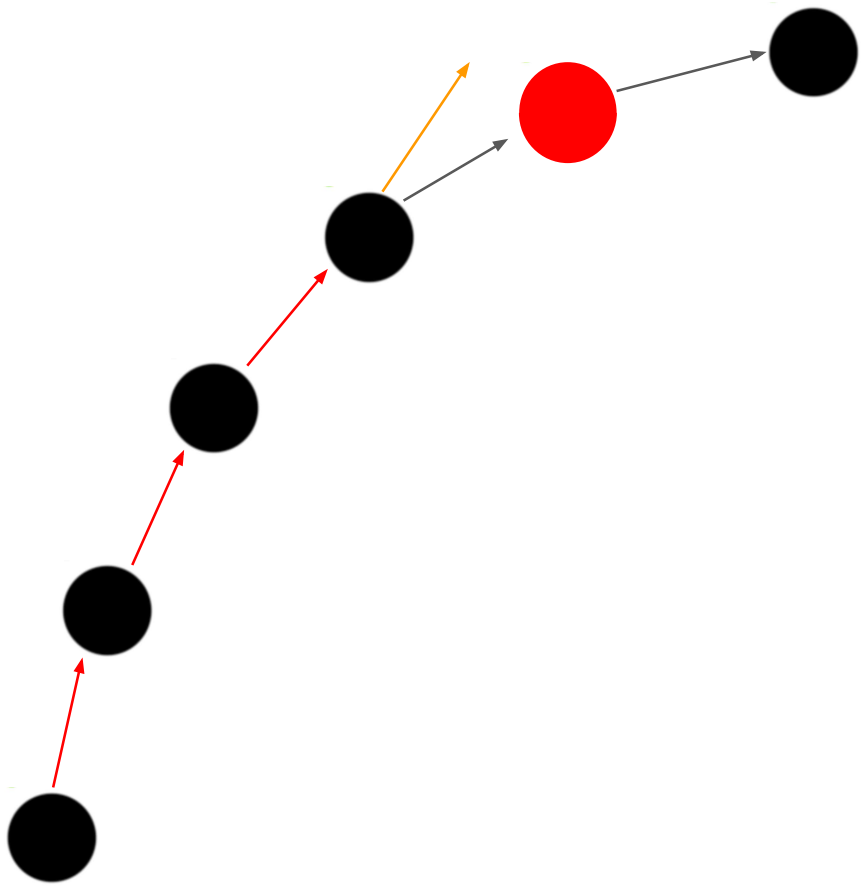


Path smoothing

Still interpolation + averaging

New steps:

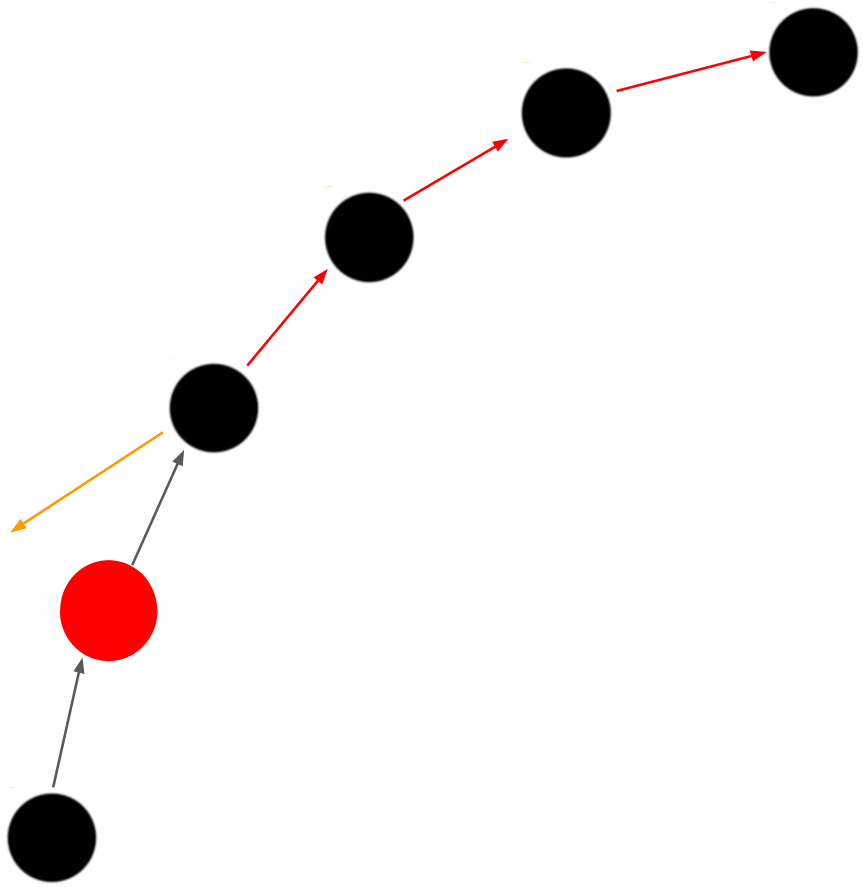
- Use the direction of the previous nodes' path
- The orange arrow is an average of the red arrows
- The new point is an average of the point at the end of the orange arrow and the red point
- Result: the end of the corner should be smoother

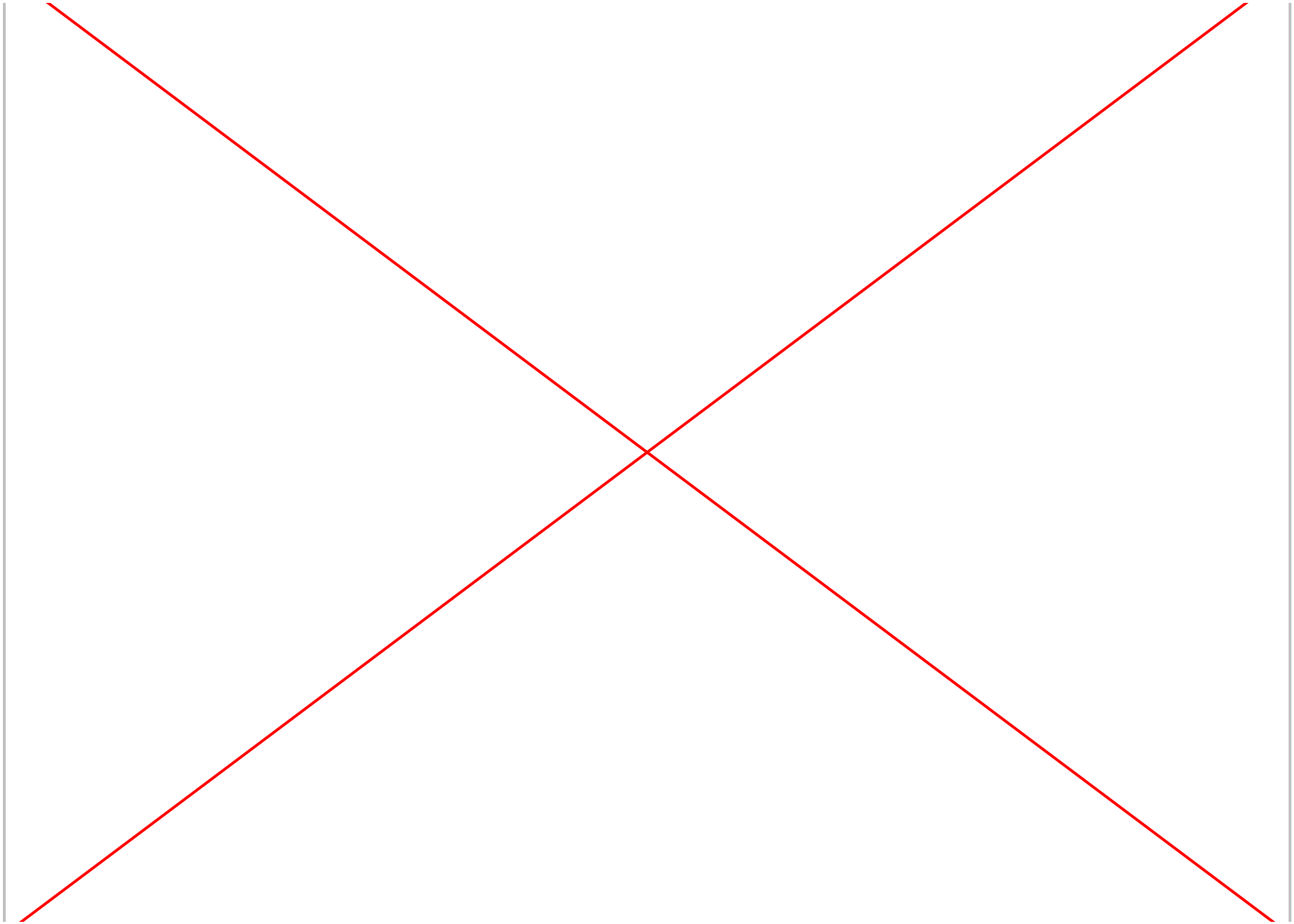


Path smoothing

Finally, do the same but with the nodes forward:

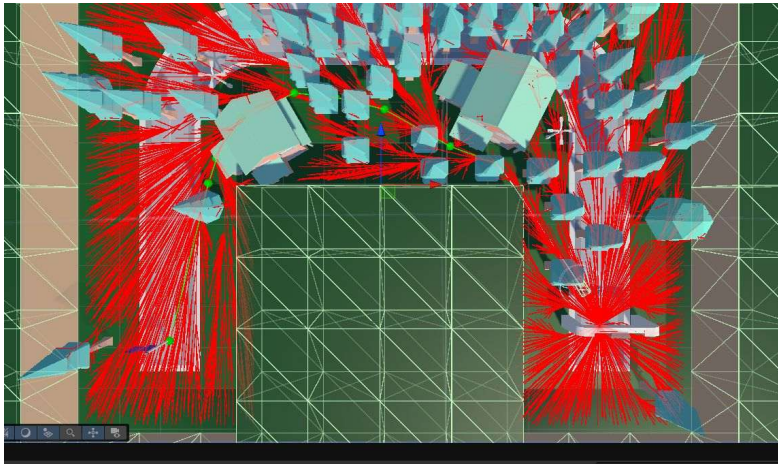
- Orange arrow = - average of red ones
- Result: the beginning of the corner should be smoother





Controller logic (Step) - Following a point

- Car looks ahead 5 nodes to find the furthest traversable point
- We also scan:
  - 18 nodes ahead for “Big Breaks”
    - → sharper turns, more breaking, limited steering
  - 13 nodes ahead for “Small Breaks”
    - → less sharp turn, less breaking, less limited steering
- Controls:
  - Clamped based on the angle to the target
  - Current problem: Brakes too much/too little!



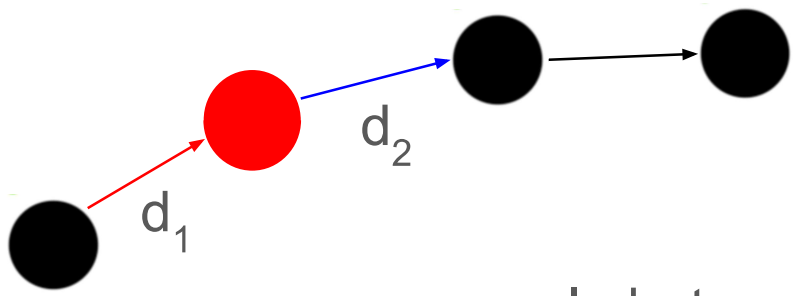
Has difficulties with this level

Controller logic (Step) - PD Tracker

Need to define a velocity to reach for each node.

Creation of a table: (Radius of curvature: MaxSpeed) us

Define the radius of curvature for each node and then th

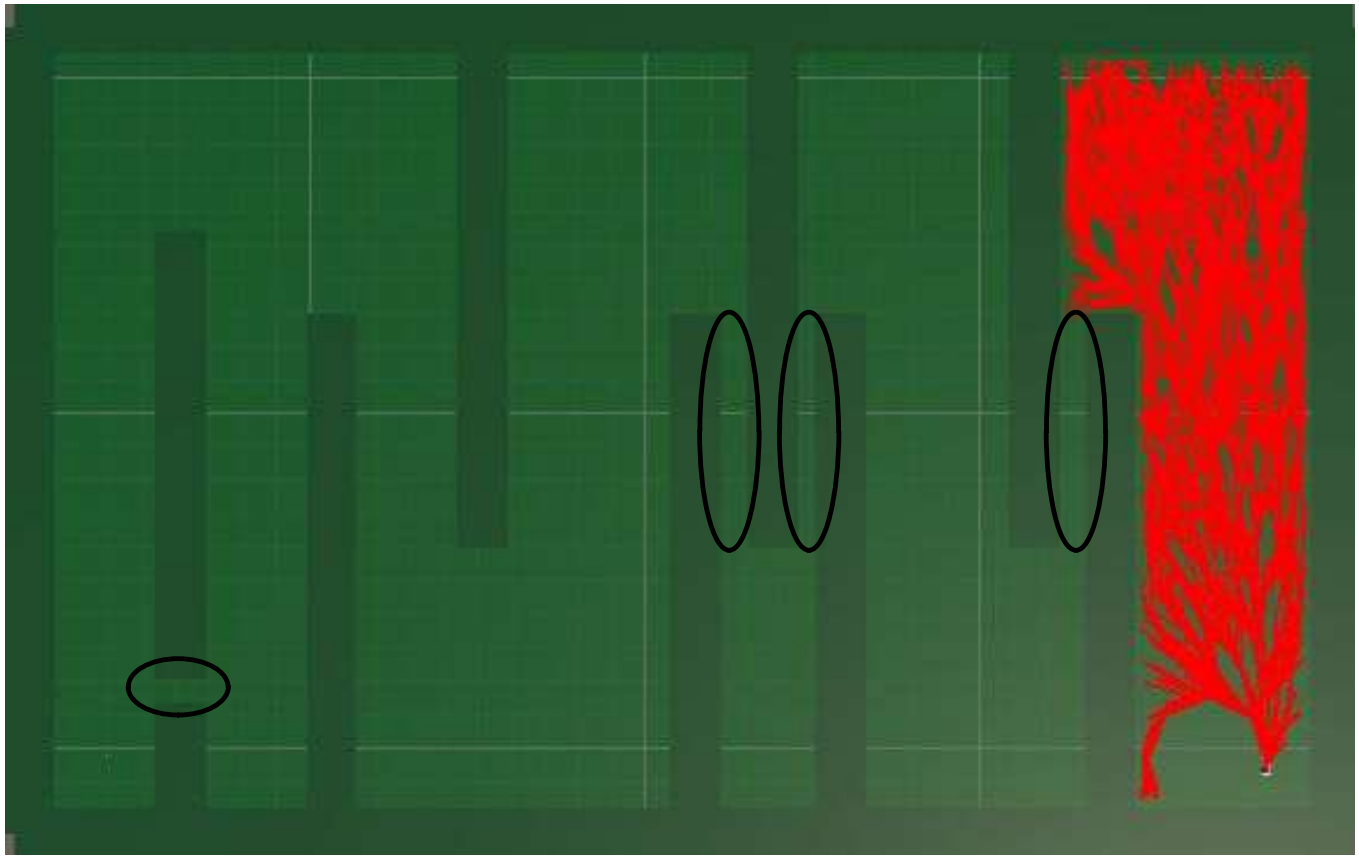


curvature = min(c

a = angle between red and blue arrows

```
// Dictionary (radius of curvature: speedMax)
var table = new Dictionary<int, int>()
{
    { 10, 18 },
    { 11, 20 },
    { 12, 21 },
    { 13, 21 },
    { 14, 22 },
    { 15, 24 },
    { 16, 24 },
    { 17, 25 },
    { 18, 26 },
    { 19, 27 },
    { 20, 28 },
    { 21, 28 },
    { 22, 29 },
    { 23, 30 },
    { 24, 31 },
    { 25, 31 },
    { 26, 32 },
    { 27, 33 },
    { 28, 33 },
    { 29, 35 },
    { 30, 35 },
    { 31, 36 },
    { 32, 36 },
    { 33, 37 }
};
```

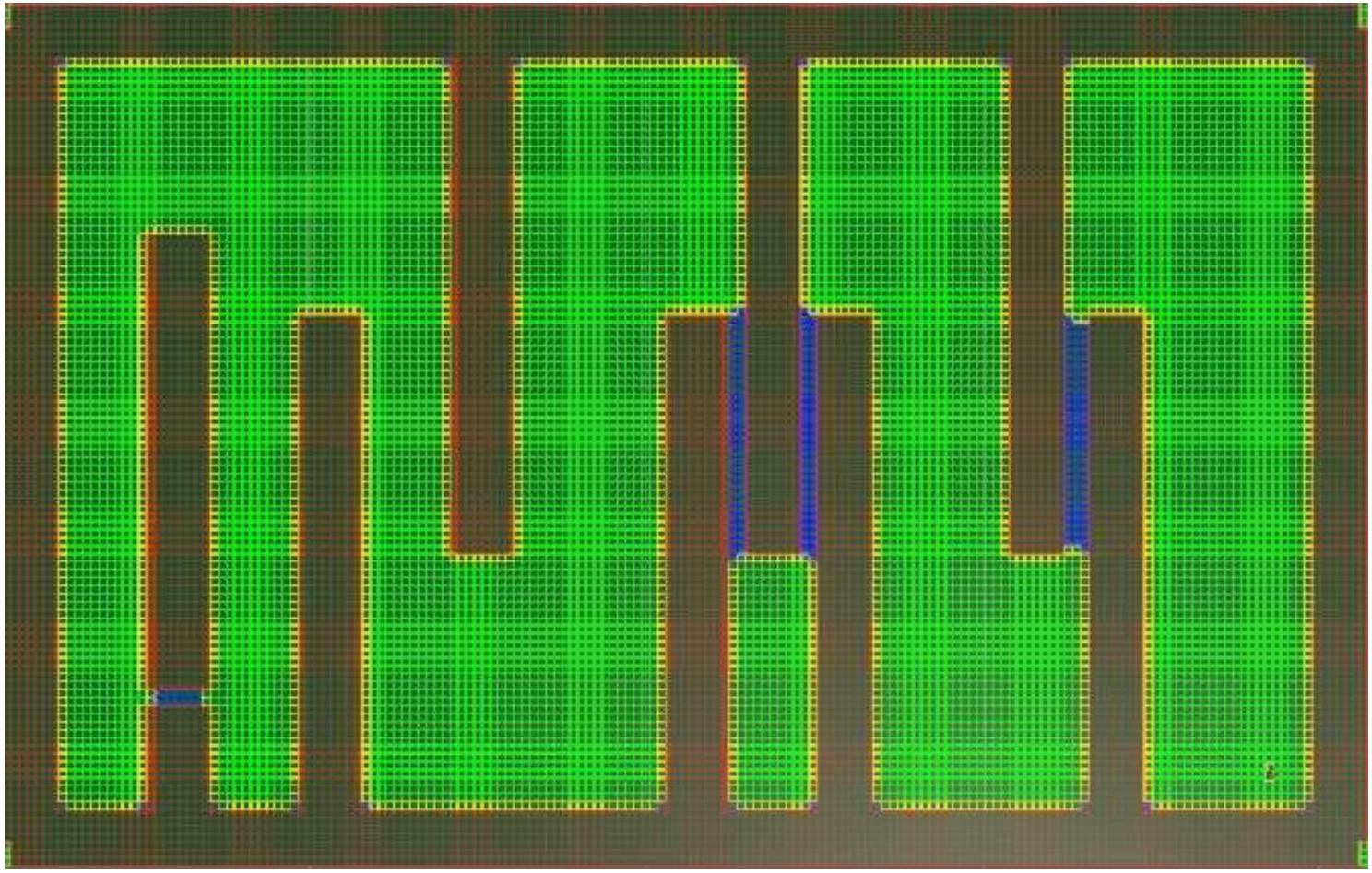
GVP RRT (Grid based Variable Probability RRT)



Small probability to pass through these narrow passages because of the randomness of RRT

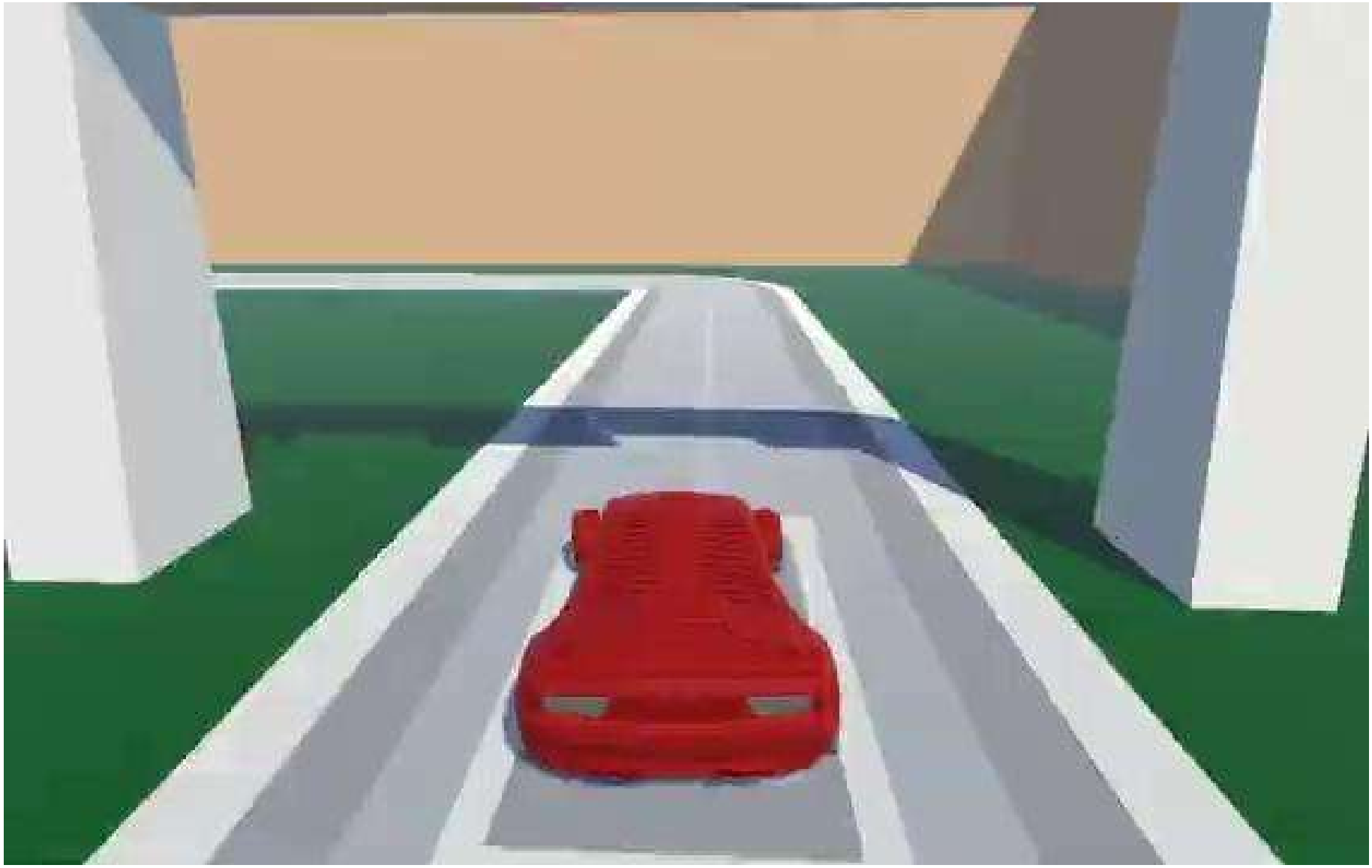
(after 40000 iterations of RRT*)

GVP RRT (Grid based Variable Probability RRT)

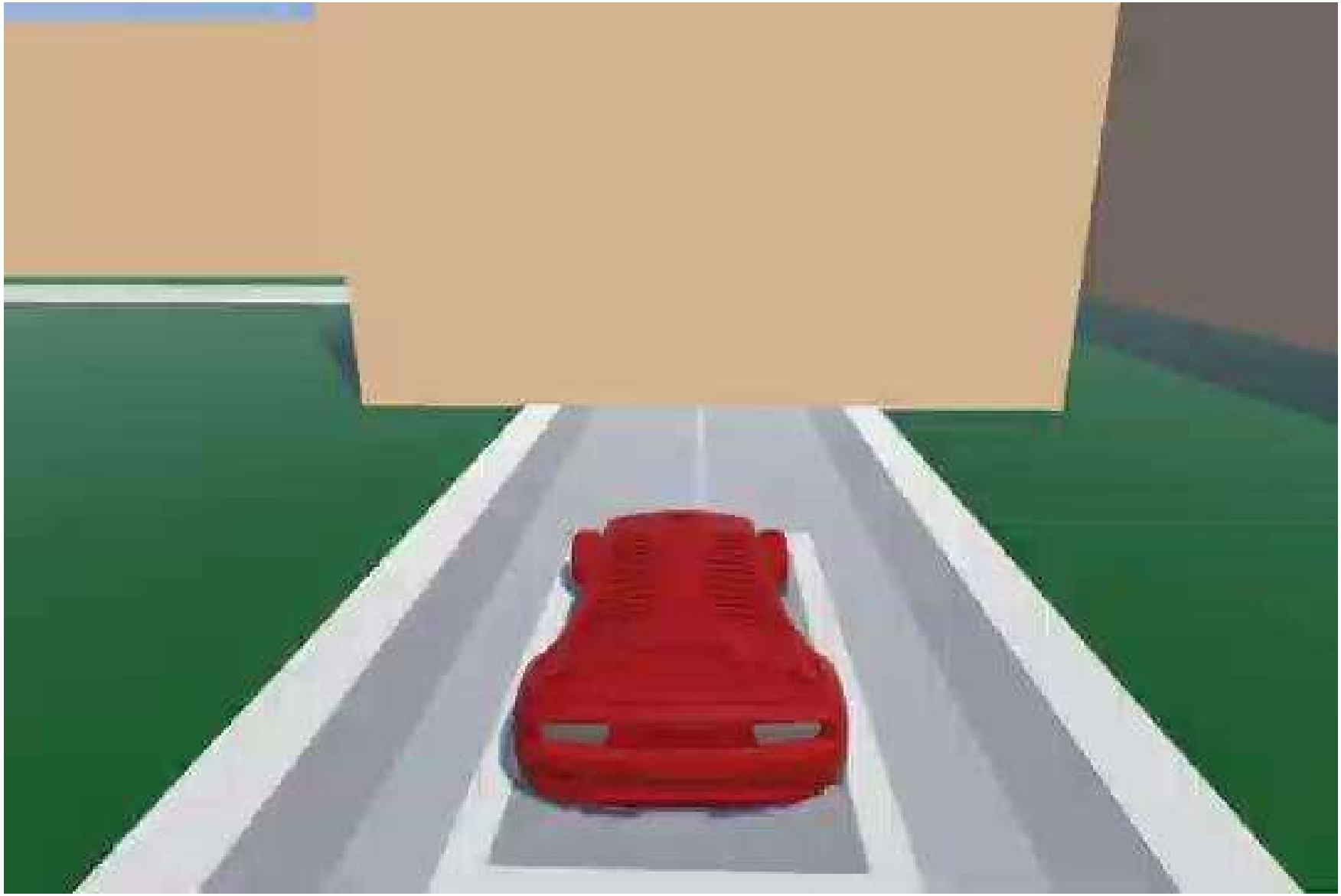


Higher probability to draw a point from the blue zones

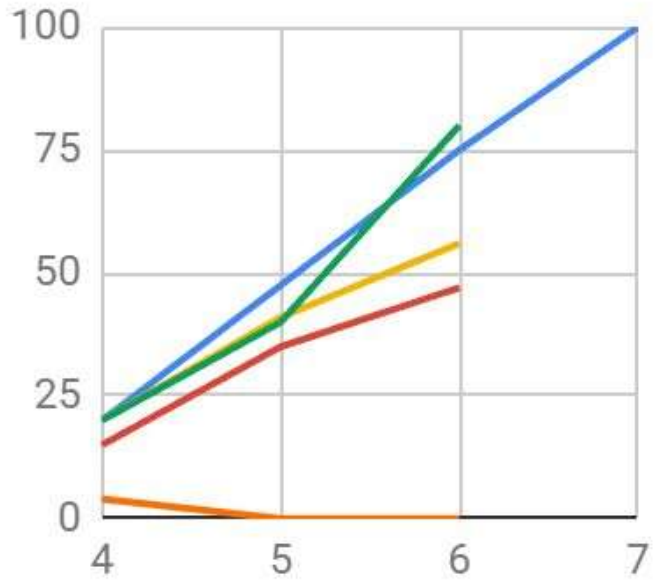
Map K2



Map K2



Map L2

Progress Status Week 6

- Comment to customer paying 250 000kr for the report:
  - CarAI is doing well and basically finished, Drone still struggling on some maps but overall good progress.
- Planned Time spent: 75%
  - Around 50 hours in total
- Actual Time spent: 53%
  - Roughly 45-55 hours per person
- Actual Progress: 80%
  - Finishing up drone and then writing the report